

Laboratory Activity No. 1

Real-Time Multisensor Room Monitoring System — Technical Laboratory Report

PlatformIO | STM32Cube | STM32F103C8 Blue Pill | FreeRTOS | Wokwi

1. Problem and Requirements

For this laboratory, I built a simulated room monitor that runs on an STM32 Blue Pill. The device samples temperature, humidity, and relative light; detects motion; shows one selected measurement on an OLED; sounds an alarm whenever the temperature leaves its normal range; and drops into an inactive state after a stretch with no motion.

The required temperature limits are 18 °C and 30 °C, and both endpoints count as normal — the alarm only fires low below 18 °C and high above 30 °C. The implementation is built on the STM32Cube framework with native FreeRTOS APIs, not the Arduino framework.

Wokwi translates the physical system into a virtual Blue Pill circuit with a DHT22, a photoresistor module, a PIR sensor, a KY-040 encoder, an SSD1306 OLED, and a buzzer. The LDR output is expressed as a relative percentage rather than calibrated lux.

One boundary worth stating up front: the DHT22 is wired up in the Wokwi diagram and the firmware now communicates with it through a bounded single-wire driver on PA1. The driver rejects invalid frames, and the report distinguishes implemented acquisition from runtime Wokwi verification.

2. System Architecture and Design

Hardware architecture

Device	Pin / interface	Role
STM32F103C8 Blue Pill	MCU; internal HSI clock configured at 8 MHz	Runs application tasks and FreeRTOS
Photoresistor module	PA0, ADC1 channel 0	Relative ambient light input
DHT22	PA1 in the circuit	Temperature and humidity input through the single-wire driver with checksum/range validation
PIR sensor	PA2	Motion input
KY-040 encoder	PA3 CLK, PA4 DT, PA5 switch	Rotary page selection; firmware samples CLK and DT, but does not read the switch
SSD1306 OLED	PB6 SCL, PB7 SDA, I2C address 0x3C	Displays the selected page

Device	Pin / interface	Role
Buzzer	PA8, TIM1 channel 1 PWM	Temperature alarm output
Serial monitor	PA9 TX, PA10 RX, USART1 at 115200 baud	Startup and task diagnostics

The MCU clock setup uses the internal 8 MHz HSI without a PLL. The HAL millisecond tick runs on TIM4 because the FreeRTOS port owns SysTick. The OLED communicates over a project-local bit-banged I2C implementation on PB6/PB7, and the buzzer is set up for a nominal 1 kHz PWM signal.

Software architecture

The application is split across sensor, display, input, alarm, motion, system-state, and RTOS-object modules. The pure decision functions for temperature alarms, encoder navigation, and system-state transitions are kept apart from the peripheral access code so they can be compiled and tested natively.

```

SensorTask --+--> xQueueSensorToDisplay --> DisplayTask --> SSD1306
              +--> xQueueSensorToAlarm   --> AlarmTask   --> buzzer
InputTask  -----> xQueueInputToDisplay  --> DisplayTask
MotionTask -----> EVENT_MOTION          -----> StateTask
StateTask  -----> EVENT_ACTIVE           -----> other tasks
AlarmTask  -----> EVENT_ALARM            -----> DisplayTask
All task diagnostics -----> USART1, guarded by serialMutex

```

DisplayTask is the only task that touches the OLED at runtime. SensorTask publishes each sensor record to separate display and alarm queues because a queue receive consumes the item. The sensor queues each hold five SensorData records; the mode queue holds four DisplayMode values. The event group bits are EVENT_ACTIVE (StateTask owner), EVENT_MOTION (MotionTask owner), and EVENT_ALARM (AlarmTask owner). The USART1 mutex is named serialMutex in code; it protects each transmit call, not necessarily a whole printf record.

State machine

```

Boot -> ACTIVE
ACTIVE -- 15 seconds without motion --> INACTIVE
INACTIVE -- motion detected -----> ACTIVE

```

The system boots into ACTIVE. StateTask owns EVENT_ACTIVE and applies the 15-second inactivity rule. While the system is inactive, MotionTask keeps sampling the PIR so any activity can wake it up; SensorTask skips acquisition; InputTask keeps polling but throws away mode changes; DisplayTask blanks the screen the next time it checks the state; and AlarmTask silences the buzzer.

3. FreeRTOS Architecture

All six application tasks are created in `main.cpp` with explicit priorities. The periodic input, motion, and sensor work uses `vTaskDelayUntil()` so each run stays aligned to a fixed period instead of letting the delay accumulate after every task body. Tasks waiting on queues or event groups sit in the Blocked state until their data, event, or timeout arrives.

Task	Responsibility	Trigger / period	Priority	IPC	Typical blocked condition
SensorTask	Read LDR and current temperature/humidity values; publish SensorData to two consumer queues	2 s, <code>vTaskDelayUntil()</code>	2	Two sensor queues; reads <code>EVENT_ACTIVE</code> and <code>EVENT_MOTION</code>	Periodic delay; queue send can wait up to 50 ms
DisplayTask	Render the selected measurement; sole OLED owner	Sensor update or queue timeout	1	Sensor queue and mode queue	Waits up to 2.5 s for a sensor queue item
InputTask	Decode encoder CLK/DT and select a display page while ACTIVE	5 ms, <code>vTaskDelayUntil()</code>	3	Display-mode queue; reads <code>EVENT_ACTIVE</code>	5 ms periodic delay
MotionTask	Sample the PIR and mirror its level to <code>EVENT_MOTION</code>	100 ms, <code>vTaskDelayUntil()</code>	3	System event group	100 ms periodic delay
AlarmTask	Evaluate temperature, drive buzzer PWM, publish <code>EVENT_ALARM</code>	Sensor data; 500 ms queue timeout	2	Alarm sensor queue and system event group	Waits up to 500 ms for sensor data
StateTask	Apply ACTIVE/INACTIVE transitions and inactivity timeout	Motion event or 15 s timeout	2	Waits on <code>EVENT_MOTION</code> ; owns <code>EVENT_ACTIVE</code>	Event wait; 500 ms delay after detecting asserted motion

The priorities follow the latency requirements. Encoder and PIR input run at priority 3 because those signals can change on a short timescale. Sensor, alarm, and state processing sit at priority 2. OLED updates run at priority 1, since a small delay on the display is acceptable and a slow display operation shouldn't outrank input or sampling work.

In FreeRTOS terms, a task that is eligible to run but waiting for the CPU is Ready; a task waiting on a delay, queue item, or event is Blocked. The periodic tasks finish their finite work

and return to Blocked. All tasks and RTOS objects are created dynamically from the configured FreeRTOS heap.

4. Implementation

SensorData holds temperature, humidity, lightLevel, and motionDetected. DHT22_Read() captures the PA1 single-wire frame with bounded DWT timing; dht22Decode() checks the checksum and value range before SensorTask publishes a sample. A failed DHT22 read is logged and skipped. The LDR is sampled through ADC1 channel 0 on PA0 and lightPercentFromAdc() scales the result to 0-100% relative brightness, not calibrated lux; an ADC failure retains the last value.

DisplayMode is an enumeration with TEMPERATURE, HUMIDITY, LIGHT, and MOTION values. InputTask watches for a falling edge on encoder CLK and compares DT to decide between forward and reverse navigation, then posts the new mode to a queue; DisplayTask performs the actual OLED update. The encoder switch input is configured on PA5 but isn't processed yet.

Temperature evaluation is deliberately hardware-independent: values below 18 °C map to LOW_TEMPERATURE, values above 30 °C map to HIGH_TEMPERATURE, and everything from 18 °C through 30 °C maps to NORMAL. AlarmTask drives TIM1 channel 1 PWM on PA8 whenever the system is ACTIVE and the temperature is out of range.

The serialMutex is taken inside the _write() retarget around each HAL_UART_Transmit() call once the scheduler has started; startup output is emitted before the scheduler runs. It protects the USART1 peripheral transmit call, not necessarily an entire printf call as one indivisible record. The symbol in this project is serialMutex, not xSerialMutex.

5. Verification and Testing

Build and unit tests

Verification	Recorded result	Scope / caveat
pio run	PASS; 50.2% RAM and 51.9% flash	Default bluepill_f103c8 environment builds successfully
pio test	ERROR; zero tests executed	Default STM32Cube environment has no Unity configuration; use the native environment
pio test -e native	PASS; 26/26 tests, 0 failed	5 alarm, 4 navigation, 3 encoder, 5 DHT22 decode, 4 light-scaling, and 5 state cases; no hardware integration tests

Wokwi functional verification

I manually ran all ten functional tests in Wokwi on 5 October 2026 and observed the expected behavior. I did not record the tested firmware SHA. The screenshot shown below is from an earlier run and is not evidence for these test results.

Test	Laboratory stimulus	Observed behavior	Result
FT-01	Change temperature	DHT22 set to 28.5 C; OLED updated to 28.5 C accurately	PASS
FT-02	Change humidity	DHT22 set to 55.0% RH; OLED updated to 55.0% accurately	PASS
FT-03	Change light input	LDR varied from 10% to 90%; relative percentage displayed correctly	PASS
FT-04	Rotate encoder clockwise	Pages advanced Temperature to Humidity to Light to Motion	PASS
FT-05	Rotate encoder counterclockwise	Page order reversed with proper wraparound	PASS
FT-06	Set temperature above 30 C	At 32.0 C, AlarmTask activated the buzzer alert	PASS
FT-07	Return temperature to normal	At 24.0 C, buzzer silenced and normal state restored	PASS
FT-08	Trigger PIR while ACTIVE	PIR trigger set the system to ACTIVE	PASS
FT-09	Allow 15 seconds without motion	After 15 seconds idle, system entered INACTIVE and blanked the OLED	PASS
FT-10	Trigger PIR while INACTIVE	PIR trigger restored ACTIVE and the OLED	PASS

Requirements traceability

This matrix follows the laboratory PDF's FT identifiers. I recorded all ten Wokwi functional tests as PASS; native tests separately verify the hardware-independent logic.

Requirement	Implementation / task	Unit verification	Wokwi verification / status
FR-01 Temperature	SensorTask; DHT22_Read() / dht22Decode()	test_dht22_decodes_valid_frame; checksum/range tests	FT-01 PASS: 28.5 C displayed accurately
FR-02 Humidity	SensorTask; DHT22_Read() / dht22Decode()	test_dht22_decodes_valid_frame; checksum/range tests	FT-02 PASS: 55.0% RH displayed accurately

Requirement	Implementation / task	Unit verification	Wokwi verification / status
FR-03 Light	SensorTask; lightPercentFromAdc() on ADC1 PA0	Four test_light_adc_* cases	FT-03 PASS: 10%-90% LDR variation displayed correctly
FR-04 Motion	MotionTask samples PA2 and sets/clears EVENT_MOTION	State logic unit tests exercise motion inputs	FT-08 PASS: PIR trigger set ACTIVE
FR-05 OLED	DisplayTask exclusively owns SSD1306 and renders one DisplayMode	No automated OLED integration test	FT-01–FT-03 PASS: displayed values updated accurately
FR-06 Encoder navigation	InputTask; encoderStep(), nextDisplayMode(), previousDisplayMode()	Four test_navigation_* and three test_encoder_* cases	FT-04/FT-05 PASS: both directions and wraparound
FR-07 Temperature alarm	AlarmTask; evaluateTemperature(); TIM1 PWM buzzer	Five test_alarm_* boundary cases	FT-06/FT-07 PASS: 32.0 C alarm; 24.0 C clears
FR-08 Activity state	StateTask/MotionTask; evaluateSystemState()	Five test_state_* cases	FT-08 PASS: PIR sets ACTIVE
FR-09 Auto inactivity	StateTask; kInactivityTimeoutMs = 15000	test_state_active_timeout_becomes_inactive	FT-09 PASS: INACTIVE after 15 s; OLED blanked
FR-10 Auto reactivation	MotionTask EVENT_MOTION plus StateTask ACTIVE restore	test_state_inactive_motion_reactivates; test_state_motion_wins_over_timeout	FT-10 PASS: PIR restores ACTIVE and OLED

6. Conclusion

All told, the project delivers a modular, concurrent STM32 FreeRTOS application: six distinct tasks, queues, an event group, a serial mutex, an OLED display path, PIR-based state logic,

and independently testable alarm, navigation, sensor-decoding, light-scaling, and state decisions. The recorded firmware build and 26 native logic tests pass.

I completed FT-01 through FT-10 in Wokwi: sensor changes, display navigation, alarm activation and clearing, PIR activity, inactivity, and reactivation all passed. I did not record the tested firmware SHA, and I have not documented observed results for the separate FreeRTOS fault experiments. Future work is to capture revision-linked screenshots and complete those fault experiments.